Quick File Navigation Guide

PORH Project

This project aims to build an automatic pipeline for PORH signal analysis. The goal is to detect key physiological events from Laser-Doppler perfusion signals and then predict PORH parameters using an LSTM model.

The main pipeline should be understood in this order:

1. Preprocessing

The preprocessing starts from the raw Excel files, one file per subject.

Main files:

- `preprocessing_script2.py`
- `data_separation2.py`

`preprocessing_script2.py` loads one Excel file, converts time values into seconds, cleans the signal, applies smoothing, and computes the signal derivative.

`data_separation2.py` applies this preprocessing to all Excel files and saves the processed outputs as:

- `_time.npy`
- `_signal.npy`
- `_params.json`

The `_signal.npy` file should contain two columns:

```
[filtered_signal, derivative]
```

2. HMM event detection

Main file:

- `batch_hmm_segmentation.py`

This script uses a Gaussian Hidden Markov Model to detect the main PORH phases:

- baseline,
- occlusion,
- hyperaemia.

It extracts three key events:

- occlusion start,
- occlusion end,
- peak time.

The detected events are saved as `_events.json`.

3. Signal alignment

Main file:

- `align_signals.py`

This script uses the detected occlusion start time to align all signals on a common reference point.

It crops each recording around occlusion start, typically from:

```
-300 seconds to +600 seconds
```

The outputs are saved as:

- `_aligned_time.npy`
- `_aligned_signal.npy`
- `_events.json`

These aligned signals are the preferred input for the LSTM model.

4. Normalization

Main file:

- `compute_normalization.py`

This script computes the mean and standard deviation of the target parameters:

- RF

- BZ
- PF
- TM
- AO
- AH

It saves these values in:

```
normalization.json
```

This file is important because the LSTM is trained on normalized target values.

5. Dataset and batching

Main files:

- `aligned_dataset.py`
- `collate_utils.py`

`aligned_dataset.py` loads the aligned signals and their corresponding target parameters. It also normalizes the labels using `normalization.json`.

`collate_utils.py` pads variable-length sequences inside each batch and returns the sequence lengths needed by the LSTM.

6. LSTM model

Main file:

- `lstm_model.py`

This file defines the LSTM architecture.

The model takes a sequence with two input features:

```
[filtered_signal, derivative]
```

and predicts either:

- 1 target,
- 4 targets: PF, TM, RF, BZ,

- or 6 targets: RF, BZ, PF, TM, AO, AH.

The current direct model uses:

```
2-layer LSTM
hidden size = 32
MAE loss
```

7. Training

Main file:

- `training_loop_full_weighted.py`

This is the main training script for the full model.

It predicts:

- RF
- BZ
- PF
- TM
- AO
- AH

The loss is a weighted MAE. The weights are computed from the standard deviation of each target in `normalization.json`.

The model checkpoints are saved after each epoch.

8. Evaluation

Main files:

- `evaluate_model_weighted.py`
- `evaluate_short_model.py`

`evaluate_model_weighted.py` evaluates the full 6-target model.

`evaluate_short_model.py` evaluates the shorter model with:

- PF
- TM

- RF
- BZ

The evaluation scripts denormalize the predictions and compute the MAE in real units.

Files to prioritize

The principal files to understand first are:

1. `preprocessing_script2.py`
2. `data_separation2.py`
3. `batch_hmm_segmentation.py`
4. `align_signals.py`
5. `compute_normalization.py`
6. `aligned_dataset.py`
7. `collate_utils.py`
8. `lstm_model.py`
9. `training_loop_full_weighted.py`
10. `evaluate_model_weighted.py`

Files that are secondary or experimental

These files are useful for reference but are not essential to the direct pipeline:

- `hmm.py`
- `training_loop.py`
- `resume_training_short.py`
- `train_tm_with derivatives.py`
- `testing_model.py`
- `dataset_creation.py`
- `data_split.py`

Important warning

There may be a mismatch between `preprocessing_script2.py` and `data_separation2.py`.

`preprocessing_script2.py` returns:

```
time, filtered_signal, derivative, params
```

but `data_separation2.py` expects something like:

```
time, signal_2d, params
```

So the next person should make sure that `data_separation2.py` builds the two-column signal correctly:

```
signal_2d = np.column_stack([filtered_signal, derivative])
```

before saving `_signal.npy`.

This is the most important point to check before running the full pipeline.